GigBid: R&D report

AI-4016 Assignment II. Engineering design review. Every number below comes from a file in `evals/results/` (named in brackets) or from a git commit. All data is synthetic (`docs/DATA_CARD.md`). Where something was not measured or did not work, it says so.

1. Problem and constraints

Workflow. Find freelance jobs, judge fit and risk, price, draft a proposal grounded in portfolio evidence, get it approved, send it, then manage the client conversation.

Never automated. Submitting a proposal, sending a message and spending Connects. Each stops at a LangGraph `interrupt()` and waits for the user in the web UI. An autonomy setting may skip low fit jobs but never sends.

Budgets, declared before measuring (`agent/config.yaml`, `budgets.declared`) and measured on the dev set with the hosted open profile [`budget_eval.json`, `proposal_eval.json`]:

Unit	Declared p95 and cost	Measured p95	Measured mean cost
job analysed (ends at decision)	20 s, 0.0005 USD	3.2 s (n = 16)	0.00006 USD
proposal drafted (to approval screen)	60 s, 0.002 USD	32.8 s (n = 18)	0.00007 USD

These are dev set numbers on a free tier endpoint, not hidden test results (section 10).

Environment. One laptop (RTX 4060, 8 GB VRAM) and free tiers: Groq (8,000 tokens per minute, 1,000 requests per day), Neon Postgres, Upstash Redis. These limits shaped every sample size below.

2. Architecture

Two durable graphs (bid, one thread per job; conversation, one thread per client) rendered from code in `docs/ARCHITECTURE.md`. The bid graph runs analyst, risk and matcher in parallel with `Send`, joins in `decide`, then pricing, writer, claim verifier, critic (at most 2 send backs), compliance, and the approval interrupt. One marketplace port has GigSim, MCP and Upwork adapters selected by `MARKETPLACE_MODE` (ADR 0001). Three of our own MCP servers (Marketplace with idempotency keys, dry run, rate limit and audit log; Portfolio, read only; Workspace) are listed with annotations in `docs/TOOLS.md`. State, execution trace and long term memory are kept apart; memory is written only by the promotion rules in `agent/memory/promotion_rules.py`. Checkpoints live in Postgres. Why each agent exists is in `docs/ARCHITECTURE.md`.

3. R&D question A: tool design

40 tasks with known correct calls, model qwen2.5:7b-instruct, nine fine grained tools against three coarse tools against the nine with rewritten descriptions [`tool_design.json`]:

Tool set	Tools	Selection accuracy (95% CI)	Argument accuracy	Tokens per task
fine	9	0.950 (0.835 to 0.986)	0.825	1008
coarse	3	0.675 (0.520 to 0.799)	0.575	710
best (rewritten descriptions)	9	0.975 (0.871 to 0.996)	0.900	1630

Coarse tools use 30 percent fewer tokens but lose 27 points of selection accuracy, which is the only difference here that clearly exceeds the intervals. Rewritten descriptions raised argument accuracy by 7.5 points for 62 percent more tokens; with 40 tasks that gain is within the noise and I would not claim it. The descriptions file was written before the first run and not edited after (recorded in the results metadata).

4. R&D question B: advanced RAG

Variants (all configurable, one command reruns): V0 dense (bge-small), V0b TF-IDF, V1 decomposition, V2 BM25 plus dense with RRF, V1+V2, V3 cross-encoder rerank, V8 corrective loop. Not built: V4, V5, V6, V7, V9.

Hypothesis (`docs/adr/0002-rag-hypothesis.md`): V1+V2+V3 raises Evidence Recall@5 by at least 10 points over V0 on jobs with 4 or more requirements. Honesty note: the ADR and the first results were committed together

(`d4bc645`), so commit order does not prove the hypothesis came first. The ADR text says it was written before the run; I cannot show that.

Result: rejected [`rag_eval.json`, n = 121 jobs, 70 with 4+ requirements]. V3 minus V0 on the 4+ bucket was +0.005 (95% CI -0.010 to +0.020). On 1 to 3 requirement jobs it was +0.053 (CI +0.032 to +0.075), the opposite of the stated expectation.

Variant	Recall@5	nDCG@10	Recall@5 (4+)	p95 ms
v0_naive	0.359	0.906	0.344	105.8
v2_hybrid	0.367	0.923	0.349	87.2
v3_rerank (best)	0.385	0.943	0.349	6994.0
v8_corrective	0.347	0.880	0.328	12738.7

Ablation [`ablation.json`]: removing the reranker costs 1.8 points Recall@5; removing decomposition costs 0.8; removing BM25 or dense costs nothing (the reranker decides the top 5). The `no_marker` row, which decomposes without the literal "Required:" line that equals the gold labels, falls to 0.355 and nDCG 0.884, below V0. So the decomposition variants partly read the answer key. This is a circularity of our synthetic data, not a general result.

Claim verification [`proposal_eval.json`, n = 18 and 17]: unsupported claims per proposal by the verifier rule fell from 0.33 to 0.00 (V0) and from 0.59 to 0.00 (V3). By the stricter audit (numbers must match evidence) V3 fell from 0.71 to 0.12, so a residue survives. The verifier checks that a cited evidence id exists, not that the sentence matches the evidence (ADR 0006).

Where better retrieval did not help. V3 had the best retrieval numbers and the lower GigSim reply rate (0.24 against 0.39 for V0, n = 17 and 18). That gap is noise. Judge scores were 3.56 and 3.59 and are **not calibrated**: the labelling step has not produced a kappa (`judge_calibration.json` shows 0 labels), so judge scores carry no validity claim. No human pairwise panel was run.

Shipped choice (ADR 0003): V2 hybrid. It is never worse than V0, costs about 90 ms against about 7 s for V3, and no measured downstream metric separates the variants on this corpus.

5. R&D question C: one agent, supervisor or graph

Same model (qwen2.5:7b-instruct, local), 30 dev jobs per architecture, policy v1 [`arch_compare.json`]:

Architecture	Bid-or-skip accuracy	Bid F1	Unsupported per letter	Expected reply probability	Tokens per job	p50 latency s
ReAct, all tools	0.233	0.34	0.64	0.220	23508	18.5
langgraph-supervisor	0.600	0.33	0.88	0.223	13113	53.8
hand built graph	0.667	0.67	0.00	0.274	1990	36.6

No architecture shipped a policy violating letter. The graph wins on accuracy, unsupported claims (it has the verifier, the others do not) and tokens (12 times fewer than ReAct). By job kind, ReAct does not beat the graph on accuracy anywhere (graph 1.0 on good, offtopic, inject, scam; ReAct 0.8 on good, at most 0.4 on every other kind), but it is the fastest. The supervisor beats the graph on lowball (0.8 against 0.0) and stretch (0.6 against 0.0) jobs, yet it scored 0.0 on good jobs, so it wins those kinds by almost never bidding, which is not a better decision rule. The graph's own weakness is that it bids on every lowball and stretch job; the minimum budget rule in section 6 removes lowballs only because of how the synthetic data is built. I would ship the graph. Caveat: the reply probability is the simulator's own diagnostic, and 30 jobs per architecture is small.

6. The change request

No concrete policy text was published for self built simulators, so I defined one ("self imposed, mirrors the announced change request format"): Connects cost x1.5, minimum job budget 50 USD, a flat 2 Connects boost fee, and no external links except portfolio links [`change_request.json`, tags `cr-start` and `cr-end`].

- **Edited:** `data/policy_pack/gigsim_policy.json` (version 2, +8 / -4 lines). A scratch file `p2.json` that a unit test writes next to it also shows in the diff (1 line, not a real policy file). Nodes read policy through `get_policy()` and the simulator enforces it generically, so **no node code changed**.

- **Added:** `tests/integration/test_change_request.py` (+64 lines, 4 tests) and a root `conftest.py` (+26 lines) that pins all other tests to a baseline policy, because eight older tests had silently depended on the live policy file (they failed after the edit until the fixture existed). That coupling is the one design cost the change exposed.
- **Time:** 3.7 minutes between the two annotated tags, 84 tests passing at the end.
- **Effect:** Bid F1 rose from 0.75 to 0.89 (ai_ml), 0.75 to 0.81 (web), 0.77 to 0.94 (data). **This is an artifact:** all 30 synthetic lowball jobs have a budget of exactly 40 USD, just under the 50 USD floor I chose, so the rule removes them by construction. It says the design absorbs a rule change cheaply, not that the decisions got better.

7. Extension X2: proposal experiments

Strategy A (evidence first) against B (outcome and timeline first), jobs assigned by hash parity, hosted qwen3.8-27b, replies drawn by GigSim [`x2_experiment.json`]. A: 5 replies of 19 (0.263). B: 3 of 20 (0.150). Difference -0.113 (95% CI -0.366 to +0.139), two proportion test p = 0.38, Fisher p = 0.45. **Not significant.** The test could only detect about a 36 point difference at 80 percent power. Planned n was 60, run n was 39 because of the request quota. Conclusion: no evidence either strategy is better. Details in `docs/EXTENSION.md`.

8. Failure analysis

Re-run of all 180 dev jobs under policy v1 (before the change request) plus the injection and proposal results [`failures.json`]. Counts by category: mispriced 30, missed evidence 24, unsupported claim 14, injection 4. Generic opening and tool error have no entries (the Critic checks openings but does not log them per job, and no tool error occurred in the dev runs).

Category	Example jobs	What happened	Fix tried	Worked?
mispriced (30)	all 30 lowball jobs, fit 1.00	bid on 40 USD jobs, the decision ignores budget against rate	none before the change request; then the 50 USD floor	yes, but only because the synthetic lowball budget is 40
missed evidence (24)	stretch jobs	fit overestimated, gold says watch, graph bids	none yet	not fixed
unsupported claim (14)	dev-web-0049, dev-data-0021, dev-web-0029, dev-data-0008	writer states past work with no valid evidence tag or an unsupported number	claim verifier before the Critic	12 of 14 fixed, 2 residual (dev-data-0008)
injection (4)	dev-ai_ml-0014, atk01, atk09, atk10	writer followed a planted instruction with defences off	delimited untrusted text, injection patterns, writer rule	3 of 4 blocked, dev-ai_ml-0014 still succeeded

Per category examples and reasons are in the file. The 24 missed evidence cases and 30 mispriced cases account for all decision errors; there were no wrong skips (bid recall 1.0) and no missed scams, but scam recall is circular (DATA_CARD limitation 1).

9. Safety and platform respect

Job posts and client messages are wrapped as untrusted data and never followed. PII masking (emails, phones, cards, secrets) is applied to logs, the audit log and LangSmith traces through one rule set. Tokens are encrypted per user. Attack success rate on 20 cases per setting [`injection_eval.json`, writer qwen3.8-27b]: **0.20 with defences off (95% CI 0.08 to 0.42) against 0.15 with them on (0.05 to 0.36)**. The intervals overlap almost entirely, so I cannot claim the defences work from this run. The writer system rule, verifier, critic and compliance banned patterns are on in both settings, so "off" is not a bare model. An attack that would beat the defences: a paraphrased instruction that the regex list misses and that asks for a harmless looking addition, such as a plausible fake credential. Live Upwork access exists only as a documented adapter stub and was never run against a real account. There is no scraping and no browser automation. Rate limiting is per user and below the simulator limit.

10. Hidden test results

The instructor runs new jobs against the live URL after the deadline. **No hidden test results exist yet** and none are claimed. Declared budgets are in section 1. The deployed link uses the hosted open profile; see the README for the live URL status. Known risks to those numbers: Groq free tier pacing (token per minute limit) adds latency under concurrent load, and the Critic rejects every draft, which spends both send backs on every proposal.

Honest list of what is not done

Kaggle datasets unused; judge not calibrated and no human pairwise panel; V4 to V7 and V9 not built; incremental re-indexing missing; Inspector screenshots taken for the Portfolio server only and without an execution result; the live Upwork and GitHub MCP paths not exercised; sample sizes 17 to 40 so most intervals are wide.